Introduction

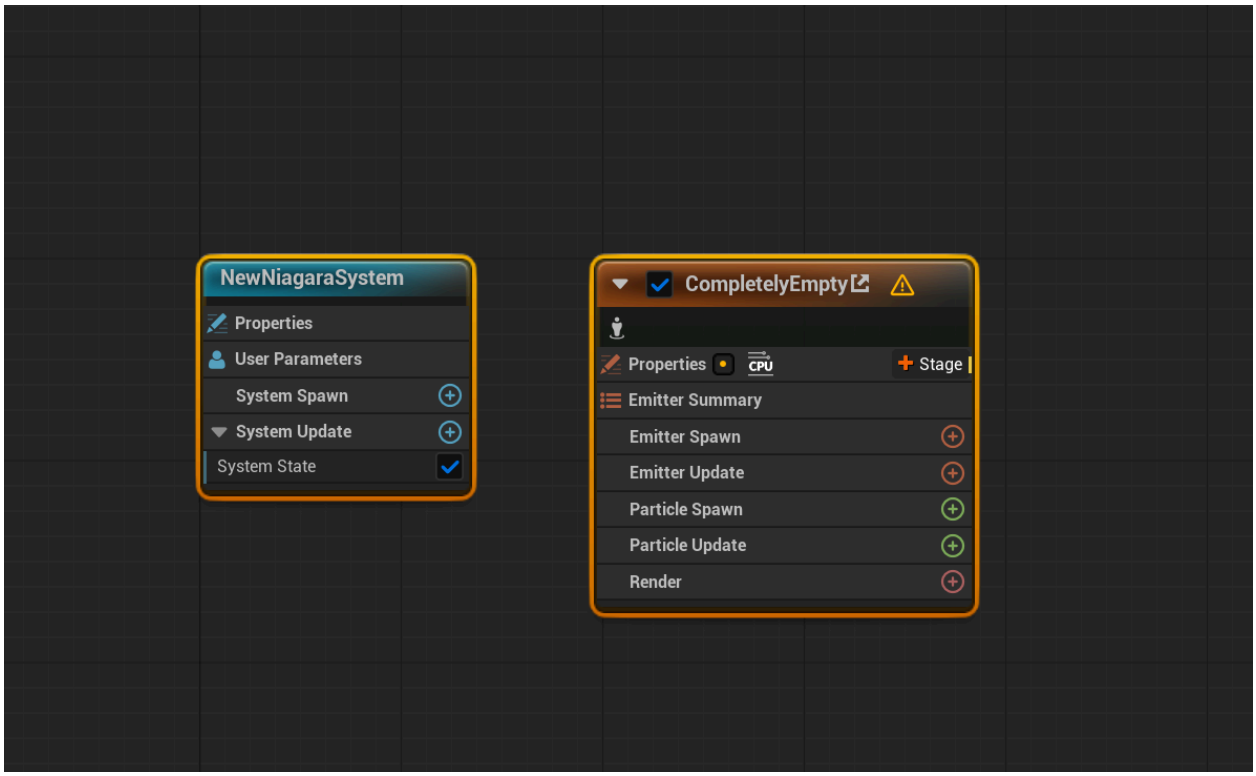
BOIDs are a neat application of computer graphics that focus on replicating the unique motion observed by groups in real life. They can be used to model the movement of schools of fish, flocks of flying creatures, such as corvids or bats, or objects that move in the wind, like leaves. In games, they can also have application in the pathfinding of enemy groups.

BOIDs achieve this life simulation through the use of 3 parameters that dictate the motion each individual unit enacts. First, with separation we ensure each unit doesn't collide with another. Second, with alignment give each unit the preference to go in the same direction as their groupmates. And finally, with cohesion we give each unit the tendency to move towards larger formations of their groupmates.

This simulation has traditionally been coded, and for those who might benefit from such an example, this pseudocode offers a decent explanation analogous to your preferred language. Luckily, Niagara systems allows us to achieve a similar simulation without the need for code (though this may be a more difficult approach for the average programmer).

Understanding the Niagara Hierarchy

First, begin by creating a new Niagara system in your file structure. Select the "CompletelyEmpty" emitter option as a blank slate to start your simulation from. Name your system however you'd like, I name my NS_Flock, and open it to begin



Immediately, we are given two blocks to work with, the system itself “[Enter your systems name]” and the emitter “CompletelyEmpty”. The important distinction here to understand in comparison with standard node systems in Unreal is the scope of the parameters we’ll be working with. By this, I mean the individual characteristic of say, a single particle, and what other systems can access it.

You will only ever have one system block per Niagara system while there may be many emitters.

For Niagara, there are four main scopes. These include the engine, the system, the emitter, and the particle. Additionally, there are two processes which have their own scopes, the stage, and the module. Both of these lists are sorted in ascending order of specificity, with the exception of the engine, which holds its unique place outside of the Niagara environment.

For reference on what a stage and module are, we can look at the bottom of the system block. “System Update” refers to a stage, which contains the module “System State”. Stages inform shared module parameters and the specific scope they refer to while modules inform more specific parameters. As you can probably guess, the prior mentioned stage and module fall under the system scope.

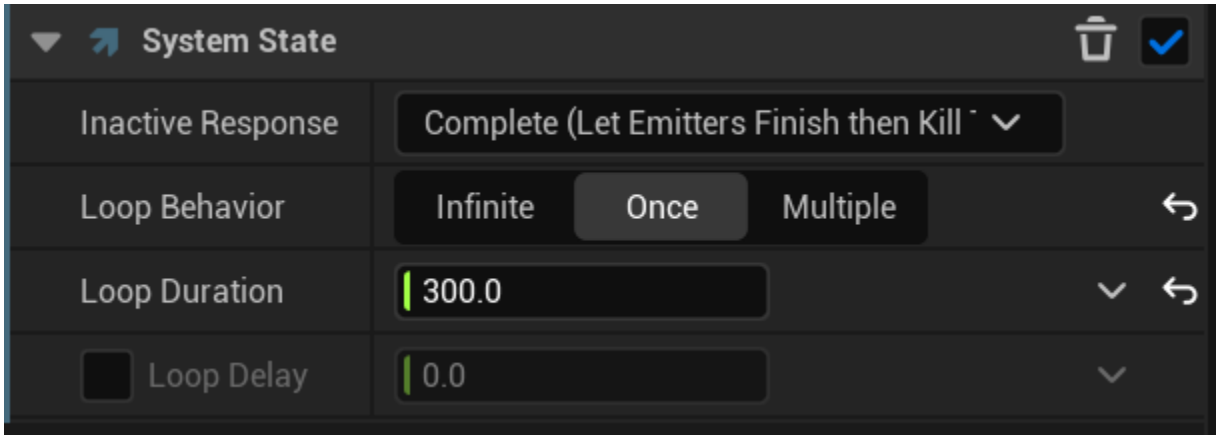
You may be wondering where the particle scope comes in, which is under the emitter block since the emitter controls the spawning of particles.

Finally, before we begin adding and modifying stages and modules, understand how the stages run. The default stages present are separated into two categories, “Spawn” and “Update”. These refer to how often their contained modules run. More specifically “Spawn” happens once upon initialization of the instance while “Update” happens every frame requested by the engine.

Initial Setup and Modification

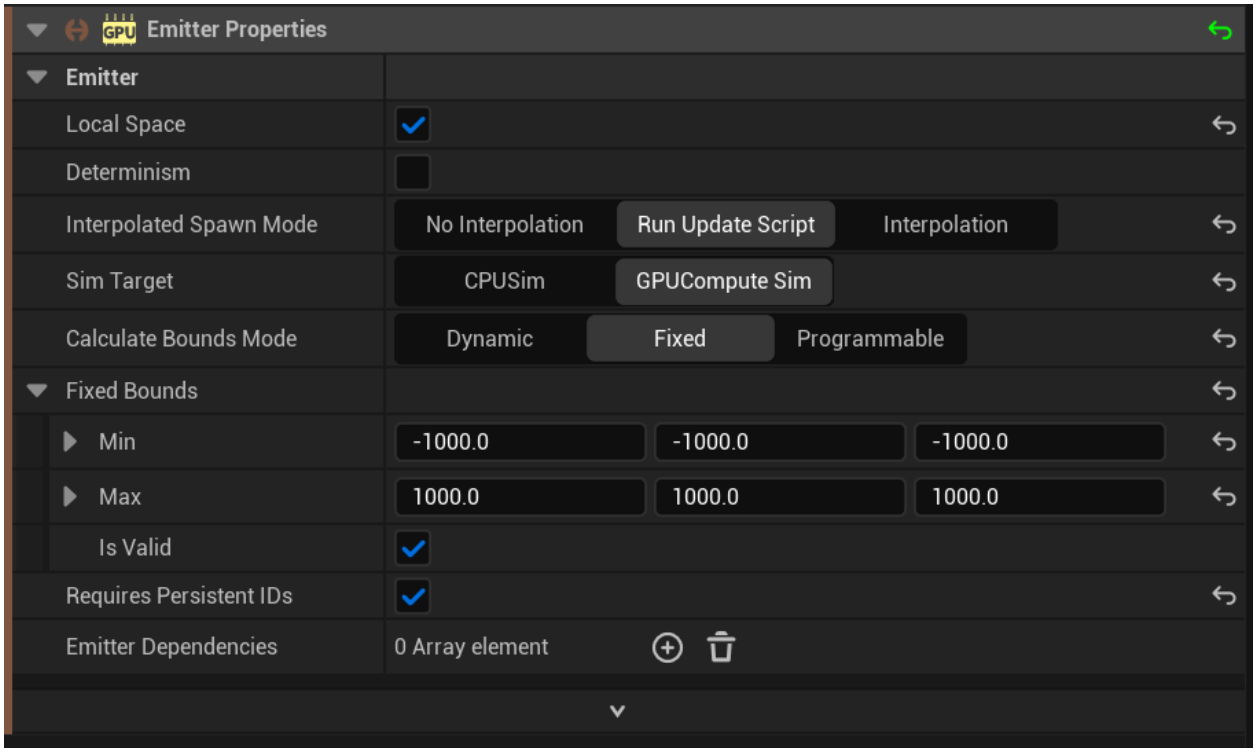
To begin, we’ll start with the simple modifications to base modules and stages.

First, open the “System State” module and swap the Loop Behavior to once, and the Loop Duration to 300.0, or 5 minutes given our game runs at 60 fps. In the end, this duration is up to you given your application of the simulation.

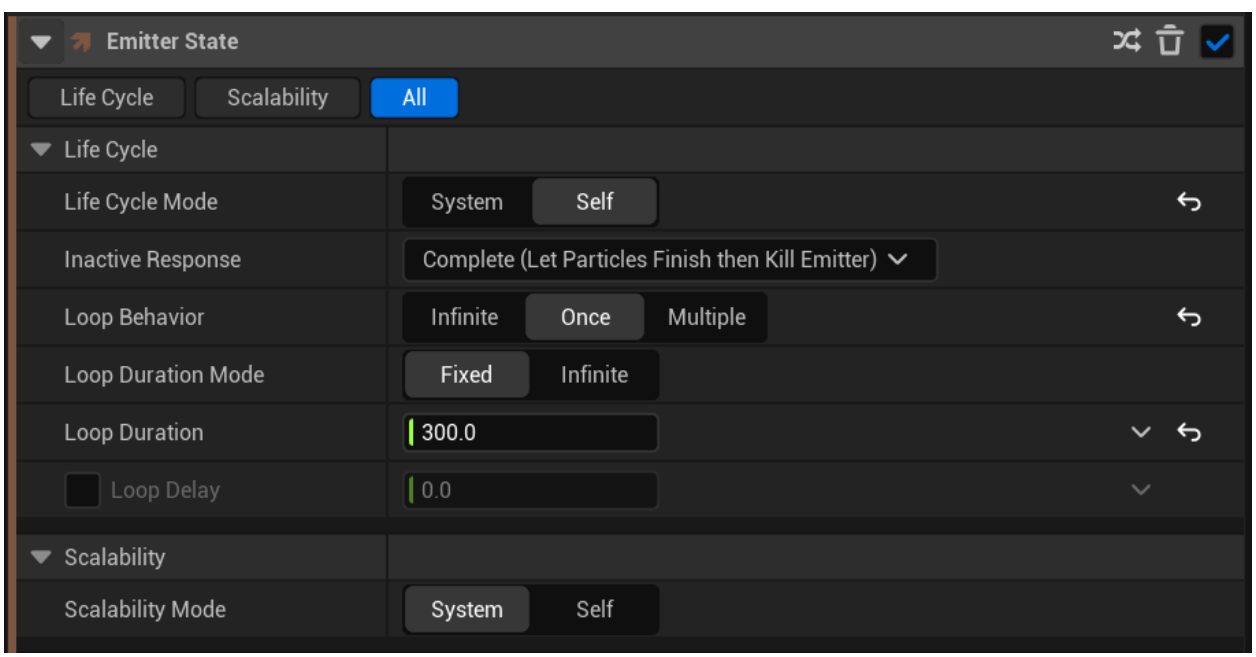


Next, open the “Properties” stage of the “CompletelyEmpty” emitter. Activate the Local Space option, which is necessary for particle position calculation. Also, swap Interpolated Spawn Mode to Run Update Script, Sim Target to GPUComputer Sim, and Calculate Bounds Mode to Fixed Bounds. All of these options ensure the system will be able to run and do the proper operations it is required to do. For the fixed bounds, I give a simple area of -1000 to 1000 for each min and max pair just to be safe.

Finally, for properties activate the Requires Persistent IDs option. This option gives each particle a consistent ID over every frame so we can easily access it in future operations.



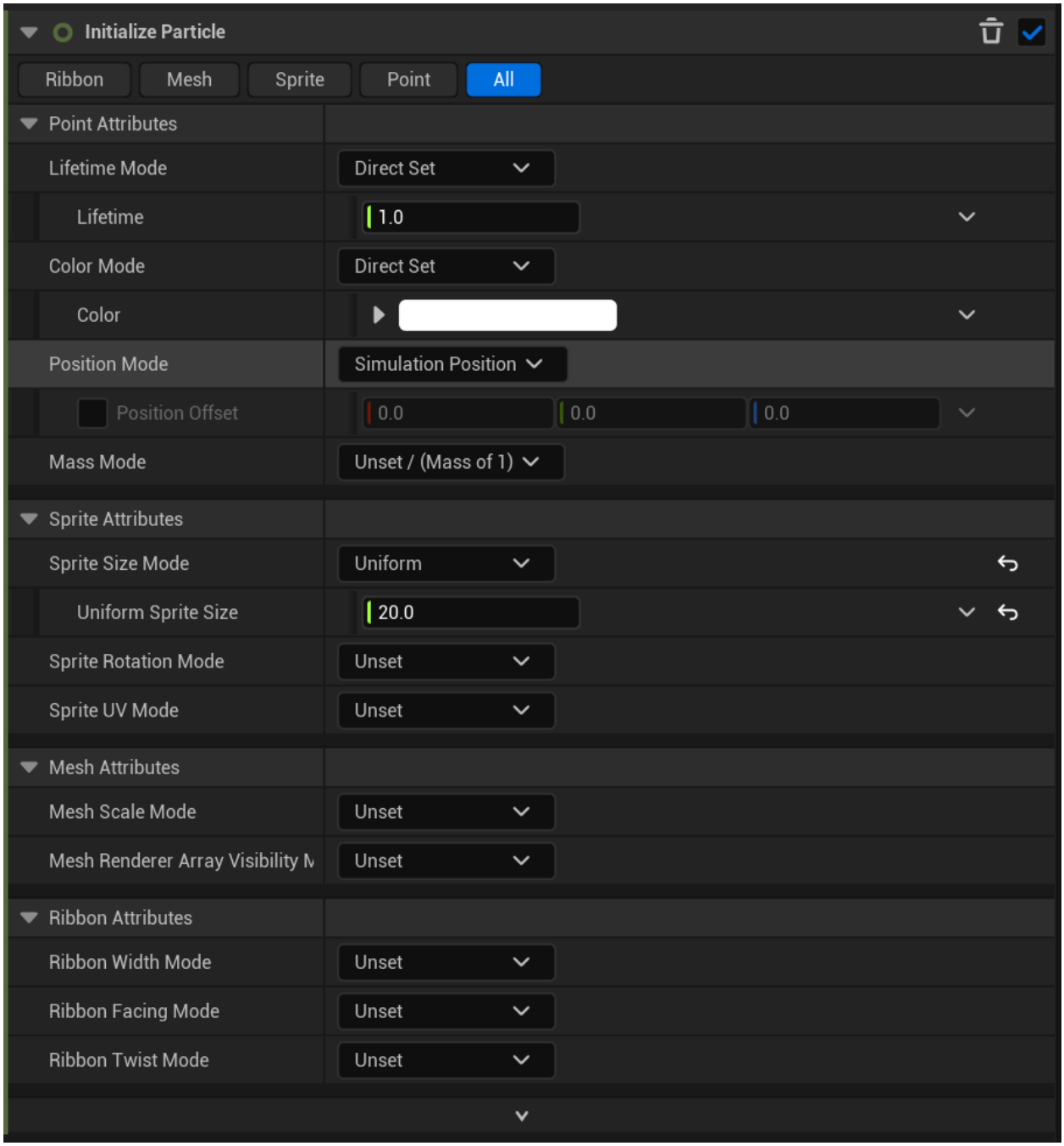
Next, in the “Emitter Update” stage click the plus and add the “Emitter State” module. This module determines how our emitter will live under our system. Set the options as follows; Life Cycle Mode as Self, Loop Behavior as Once, and Loop Duration as 300.0, similar to the “System State” module.



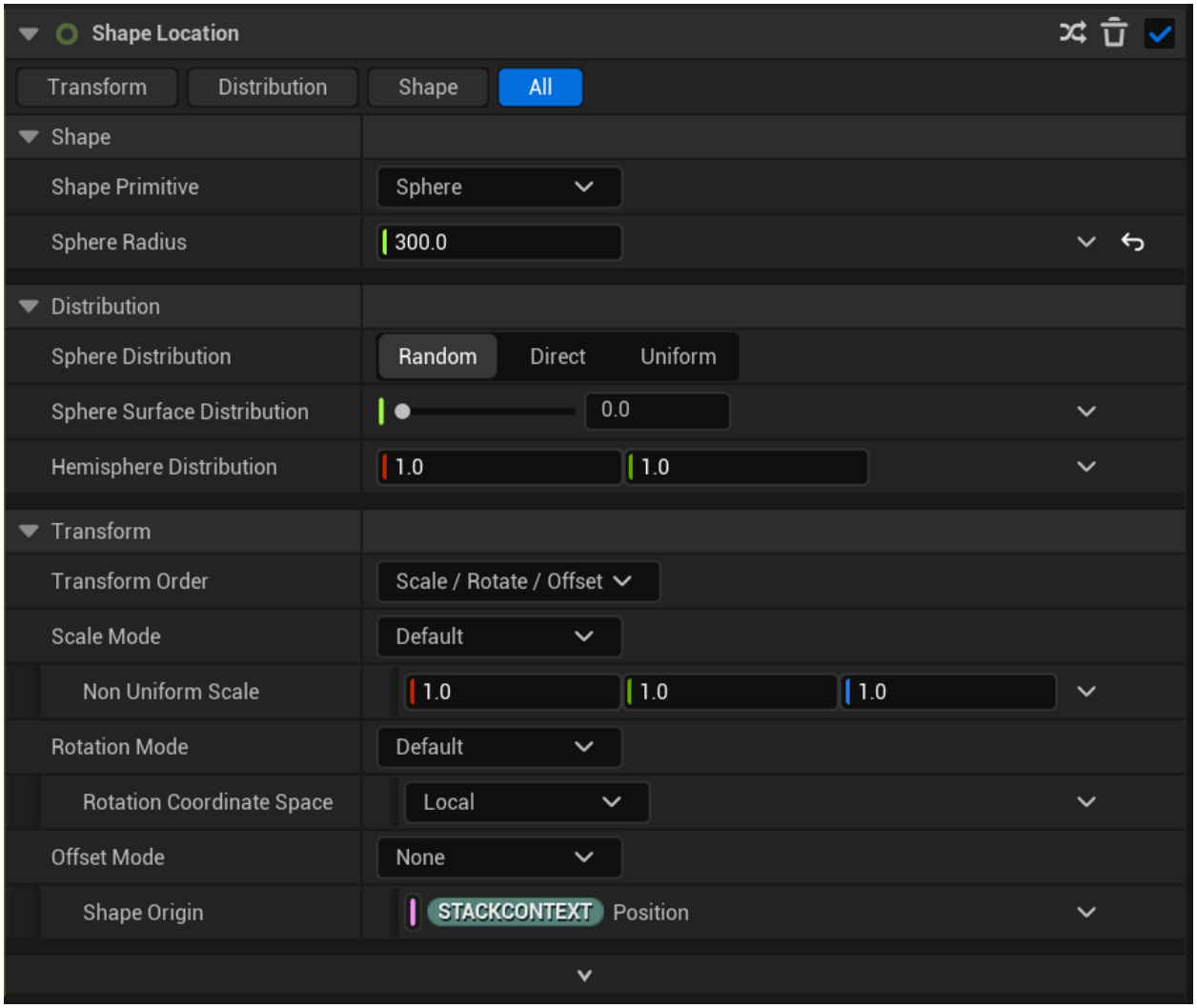
Next, add another module to the stage with the plus, this time a module called “Spawn Burst Instantaneous”. This module allows the emitter to create a lot of particles all at once at a specific time. The only parameter we’ll need to change here is Spawn Count, which for my example is 100. This is a major area of customization however, as different simulations will require different amounts. For example, a school of fish may only be 10 to 15 particles or some buzzing flies may only be 3 to 7. It’s entirely dependent on what you’re attempting to replicate.

Moving on to the “Particle Spawn” stage, we’ll be adding two modules, “Initialize Particle” and “Shape Location”. Do note that it's important that modules are added in order as they run in that order.

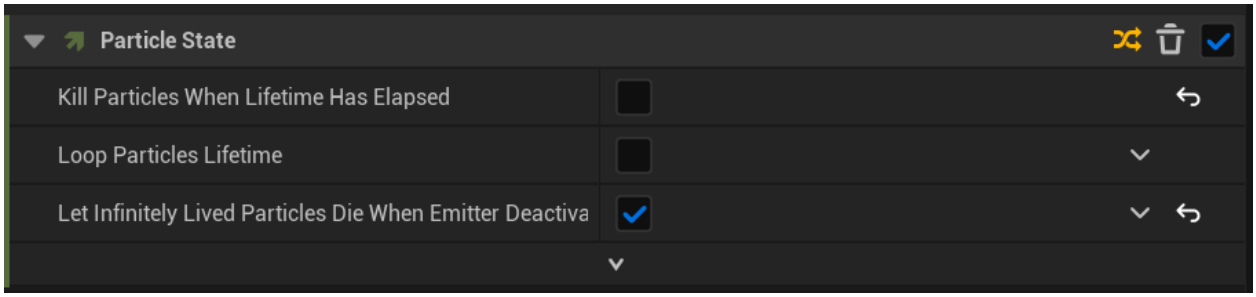
For “Particle Spawn” the default options will suffice apart from Sprite Size Mode under “Sprite Attributes”. Set it to Uniform and the resulting float beneath it to 20.0. This is for the size of each of our particles when we get to rendering, specifically sprite rendering. If you’re planning an alternative renderer, please ignore.



For “Shape Location” set the Shape Primitive to Sphere and the resulting radius below to 300.0. This module allows our particles to spawn in random locations with that defined sphere.

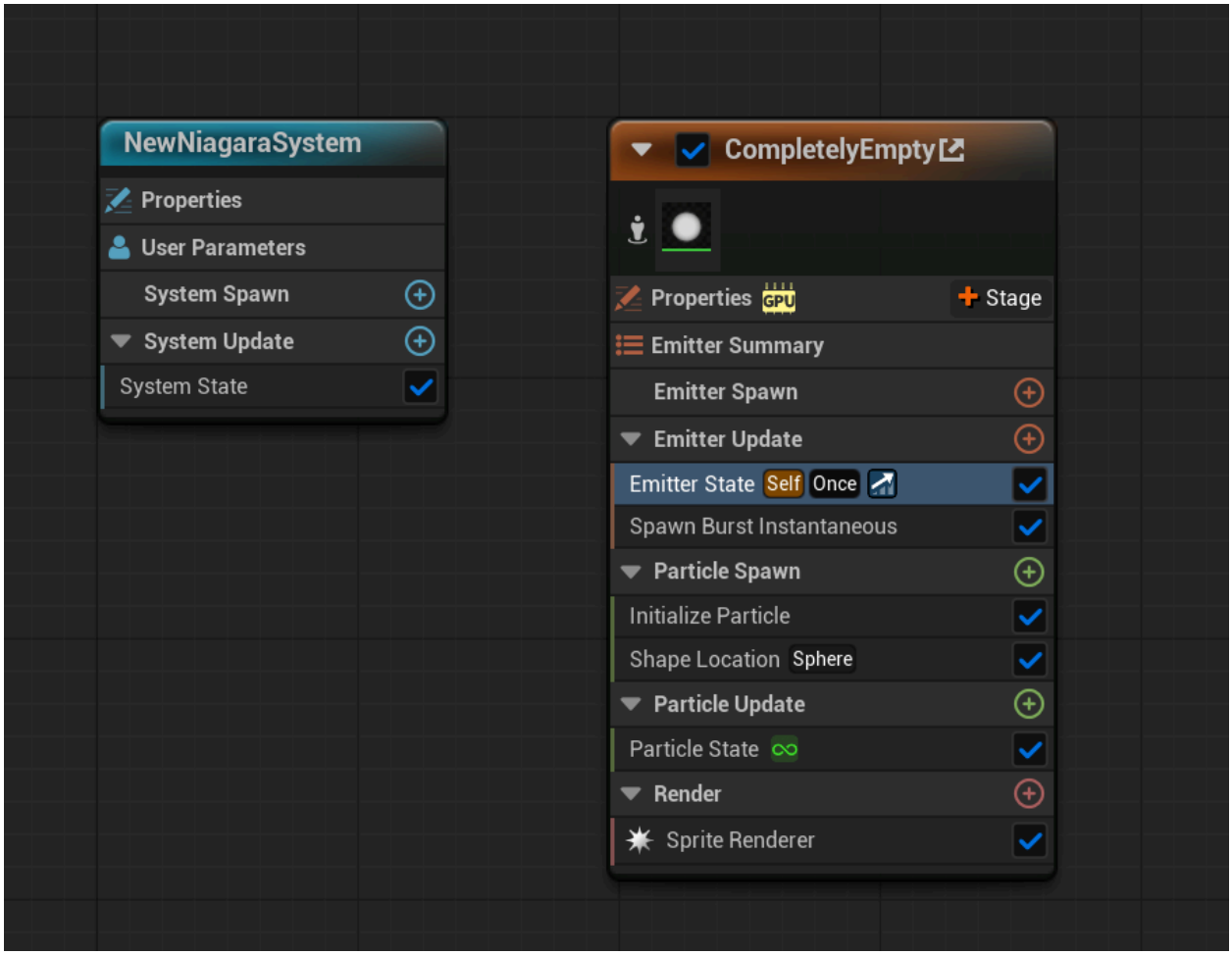


Another default module can be added to the “Particle Update” stage, specifically the “Particle State” module. For this, uncheck the first option, Kill Particles When Lifetime Has Elapsed, and check the third option, Let Infinitely Lived Particles Die When Emitter Deactives. This keeps our particles around forever until we remove the system and therefore the emitter.



To conclude our default module, we can finally move onto the last stage “Render”. Here, add the module “Sprite Renderer”. All of its defaults are appropriate for copying my example, but this is where specific visuals would be applied, like petals for example.

At this point in the process, your emitter should look like the following:



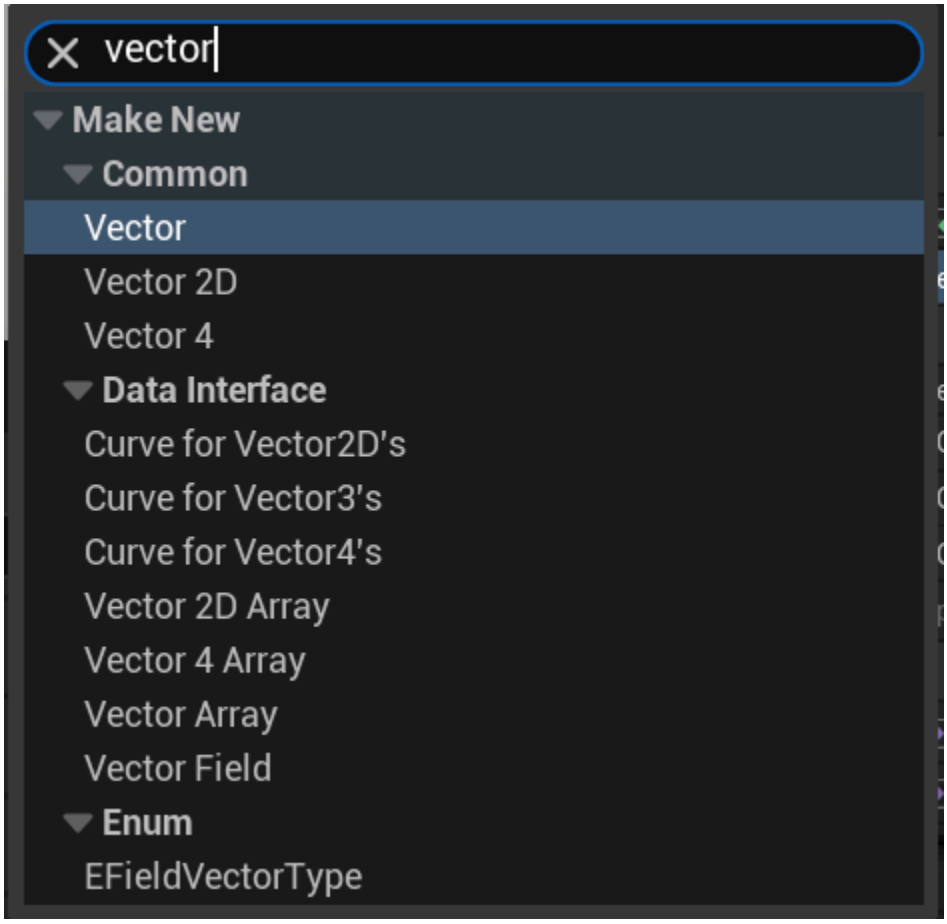
Custom Parameters and Modules

To begin creating our custom modules that enable our particle's motion, we need to make some unique parameters that we can manipulate. We can do this by looking at the "Parameter" window of the editor. If you're using the default layout, this should be the first of three tabs present beneath the preview window on the left side of the window layout. If the window isn't present, open the "Window" drop down at the top navigation bar of the editor, it should appear in the first few options there.

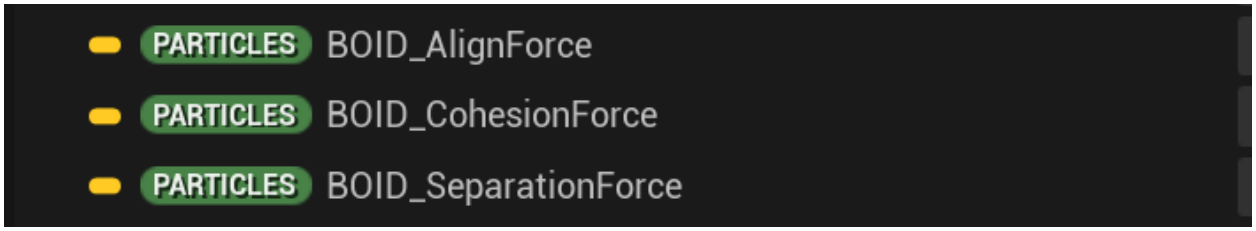
Within the parameter window, we need to move down to the "Particle Attribute" section. This section holds the parameters specific to the particle scope, which you can see denoted by the green PARTICLES tag before each parameter's name. Other than that, we can see a few other signifiers which tell us different things about the parameter. The colored notch to the left of the particle tag depends on the parameter's type, similar to what colors we see in the modules. In the box on the right, there's a lavender number and green number, which tell us how many times the parameter is read and written respectively. Finally, for some parameters there is a little lock next to our reference numbers, which lets us know whether the parameter is referenced in a graph and therefore cannot be edited drastically within this window.

Now that we understand how these parameters present, let's make a few, specifically the accumulators needed to store our forces, separation, alignment, and cohesion. Click the plus to within to the right of the "Particle Attributes" header.

This will open up a search, which for each of our three forces we'll be using the vector option.

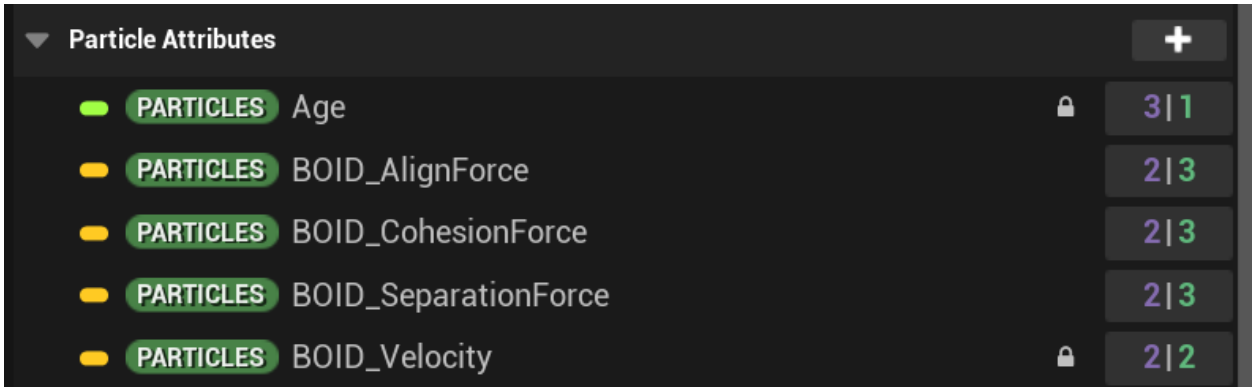


This creates us a new parameter called "Vector", double click on it to rename it and enter the names as seen in the image below.



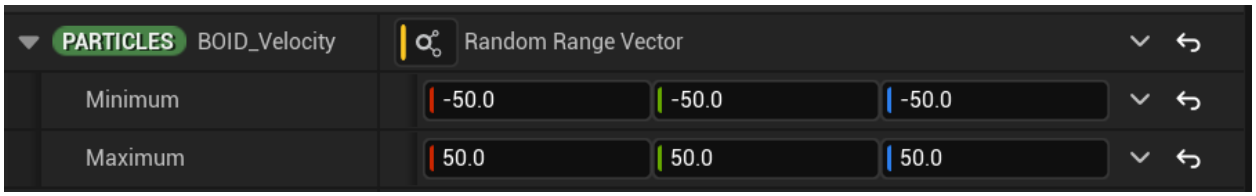
We need one final vector parameter to hold our custom velocity, since the standard velocity parameter defined by the system causes unintended effects elsewhere. Create another vector parameter the same as above and name it BOID_Velocity. Your “Particle Attributes” section should look like this at the top now:

Note that your reference numbers will be far different for now.

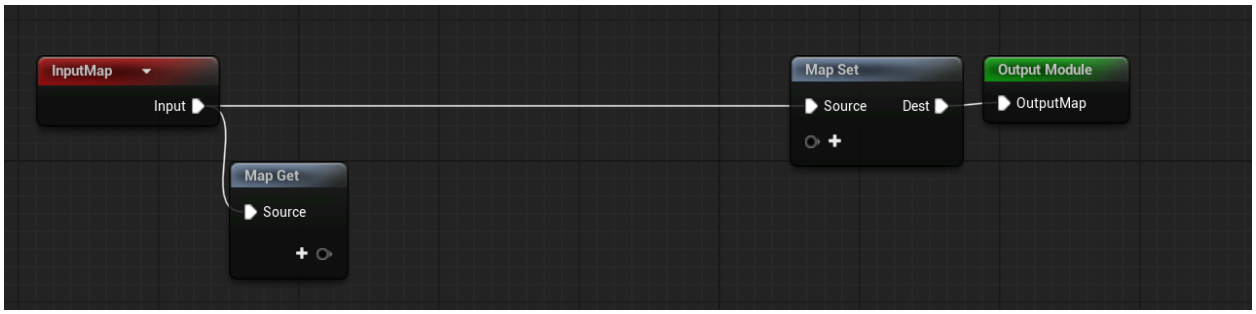


Next, to prepare these parameters in our emitter, add a new module to the “Particle Spawn” stage. Search for “set” and select the “Set new or existing parameter directly” module. Open the module and add our newly created vector parameters to it. This can be done either by dragging them from the parameter window on top of the “Set Parameters” header or by clicking the plus to the right of that header and searching for the desired parameter.

With this setup, we should be fine, but I like to give the particles a little boost to get started. To give them a random velocity, open the drop down next to the BOID_Velocity’s individual floats and search for “Random Range Vector”. Within the minimum and maximum pairs that follow, set them to -50.0 and 50.0 respectively.



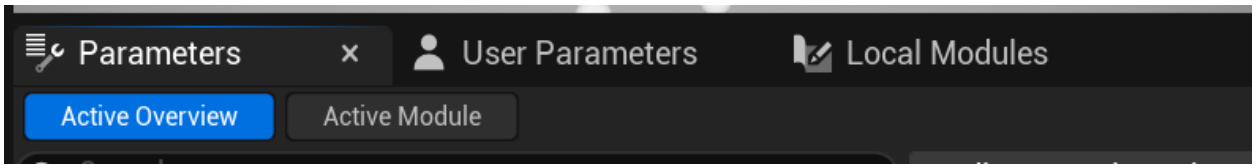
Now we'll be creating our first scratch pad module, a custom module that allows us to create our own node graph. Add a new module to the "Particle Update" stage, specifically the "New Scratch Pad Module" module. This should open a new tab next to the system overview tab and should open the local modules tab where the parameter tab is, right click on the "ScratchModule" option there and rename it to "Reset_Accumulators." Within the graph, you should have four nodes.



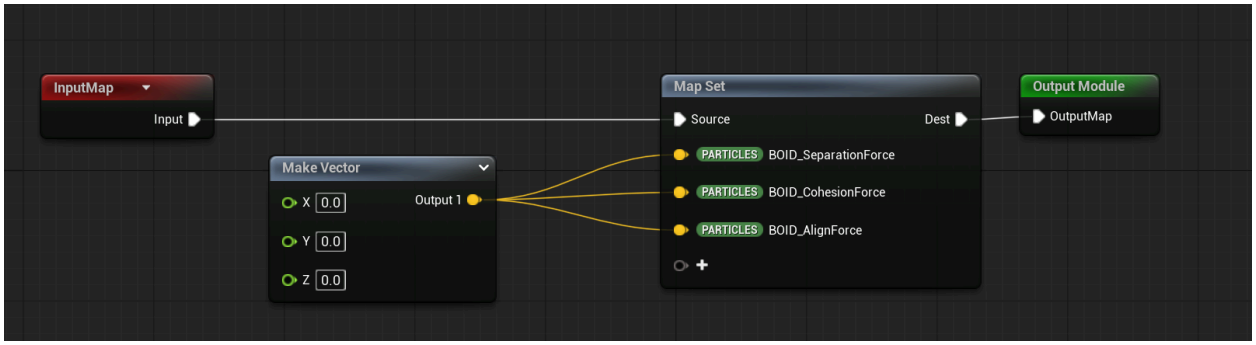
The red InputMap and green Output Module nodes tell us where the start and end of our graph lie. The Map Get and Map Set nodes are where our operations start and end, retrieving our parameters that we need and setting the values of the parameters respectively.

For this "Reset_Accumulators" module we don't need the Map Get node, feel free to delete it. Right click on the open graph and add a "Make Vector" node:

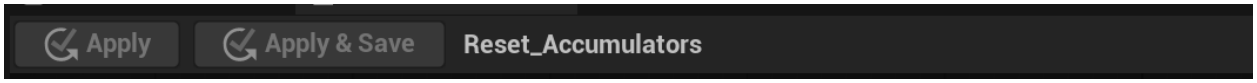
This node is fine as default. Now to get the parameters we need to set, open the parameters window again. Currently, it should be empty, this is because the "Active Module" option is selected showing only the parameters used in the graph. Select the "Active Overview" option and drag all of our custom vector parameters onto the set node except for BOID_Velocity.



Wire the "Make Vector" node into all three vector parameters. Your final graph should look like this:



At the top of the graph window, click the “Apply and Save” button and return to the system overview.



Custom Stages

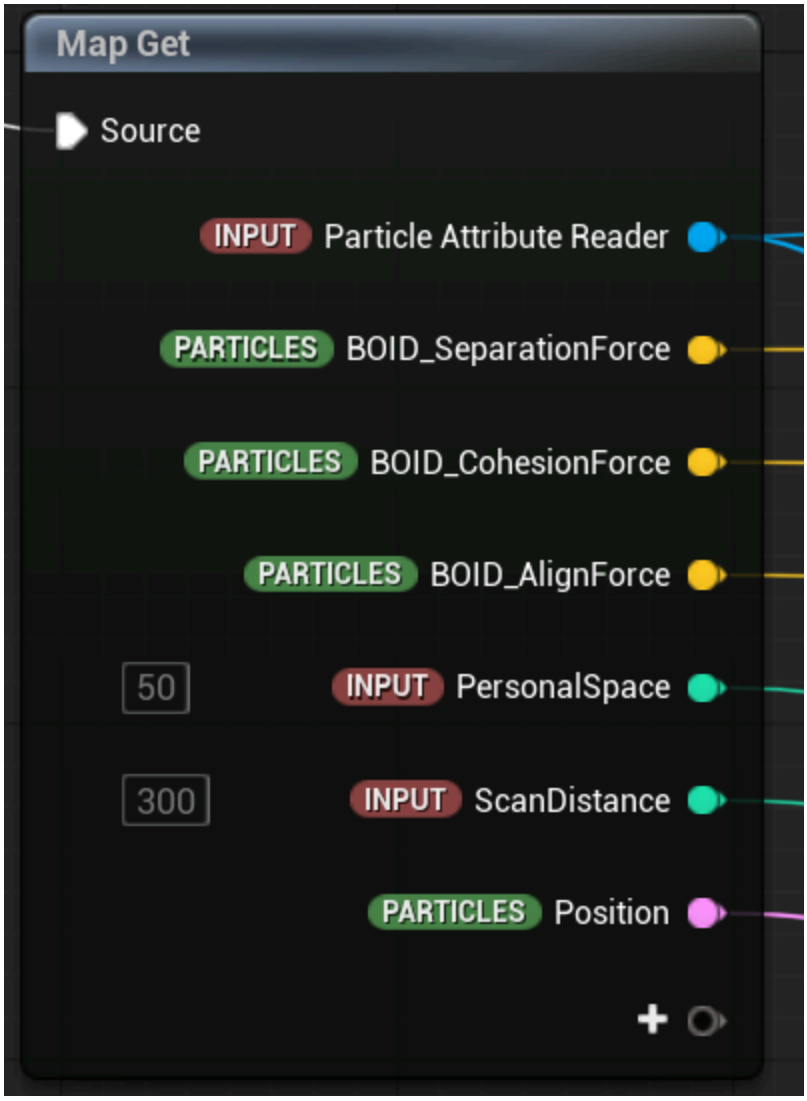
Now, in order to keep our emitter organized, we’ll be creating a custom stage. At the top of the emitter block click the “+ Stage” button and select the “Generic Simulation Stage Settings” option in the search. This will add a new “None” stage below the “Particle Update” stage. Open the header and rename it how you see fit, I name mine “Boids”. The rest of the settings should remain default.

In order to keep the remainder of this tutorial concise, I will assume you understand how to search for modules, manipulate the parameters window, and wire nodes together. I’ll keep explaining new concepts and providing images.

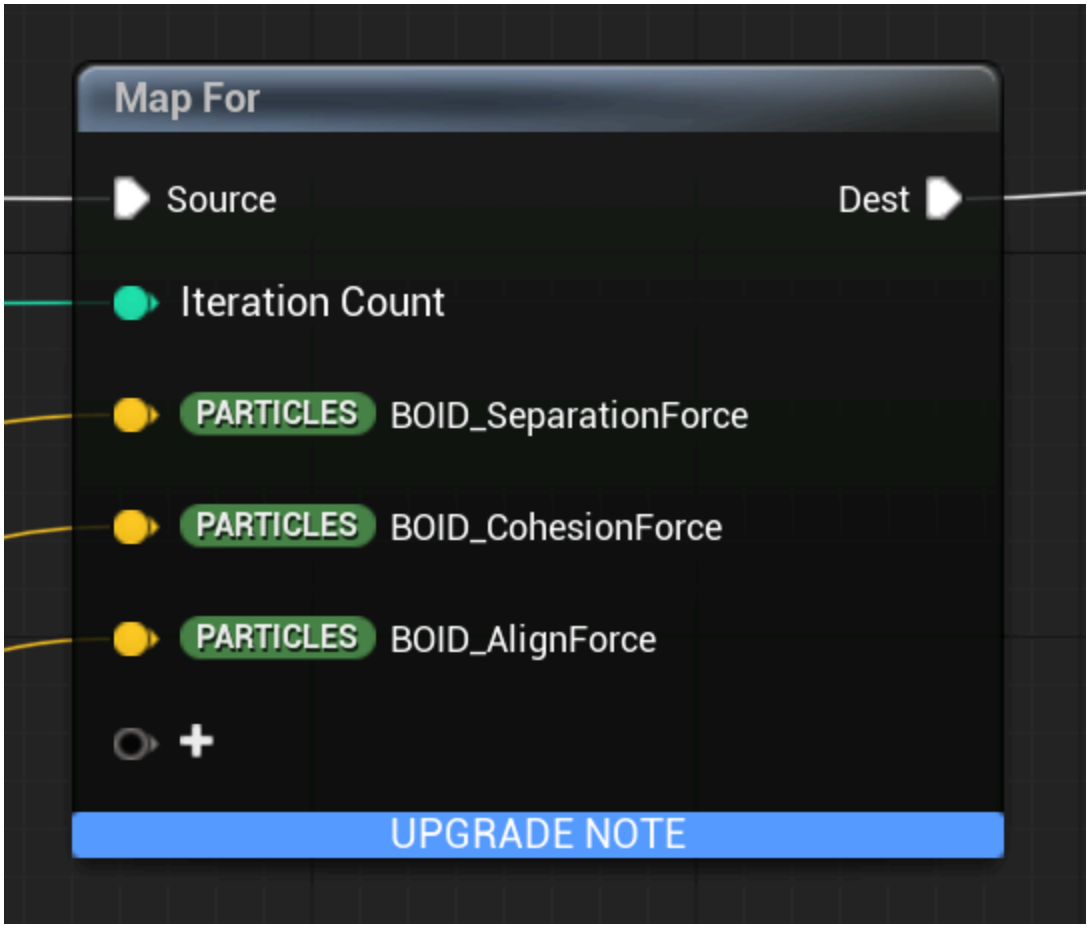
Create a new scratch module under this new stage. Name it “BOID_Sum_Forces”. This module will do the heavy lifting of determining the forces acting on the particles. Swap the Map Set node for a Map For node. This node is the same as a Map Set node, but it runs the operations prior to it as many times as the “Iteration Count” integer parameter says.

Within the parameter window, add three parameters to the “Module Inputs” section. These parameters will be visible from the system overview tab, allowing you to modify the system quickly on the fly. Add one Particle Attribute Reader and two int32’s named “PersonalSpace” and “ScanDistance”.

Drag these three inputs, the three accumulator vector parameters, and the default particle “Position” parameter into the Map Get node. Within the parameter window open PersonalSpace and ScanDistance. Set their default values to 50 and 300 respectively.

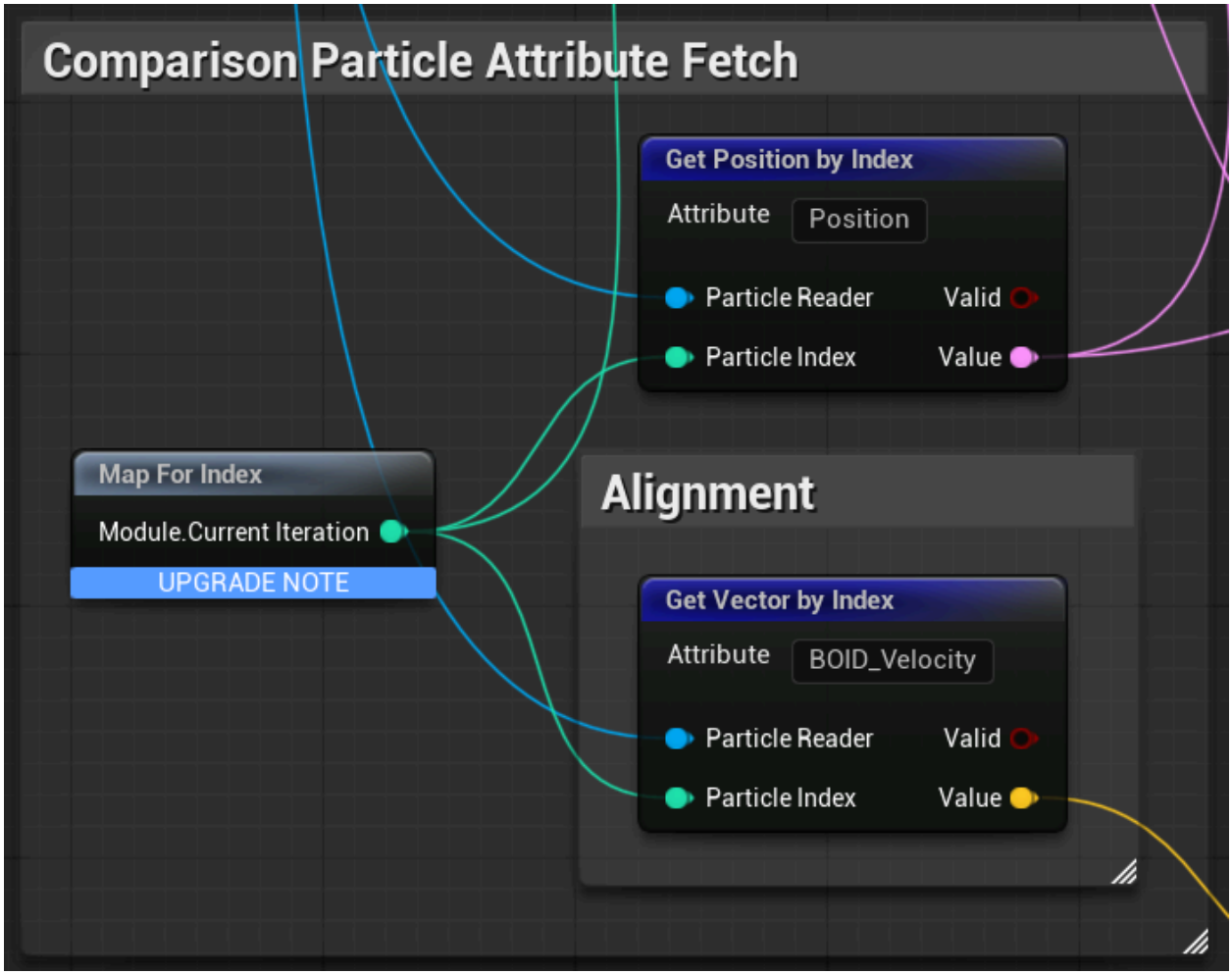


Drag the accumulator vector parameters on the Map For node also.



While wiring off of the Particle Attribute reader, add a “Get Num Particles” node and wire it’s integer output in the Iteration Count of the Map For Node.

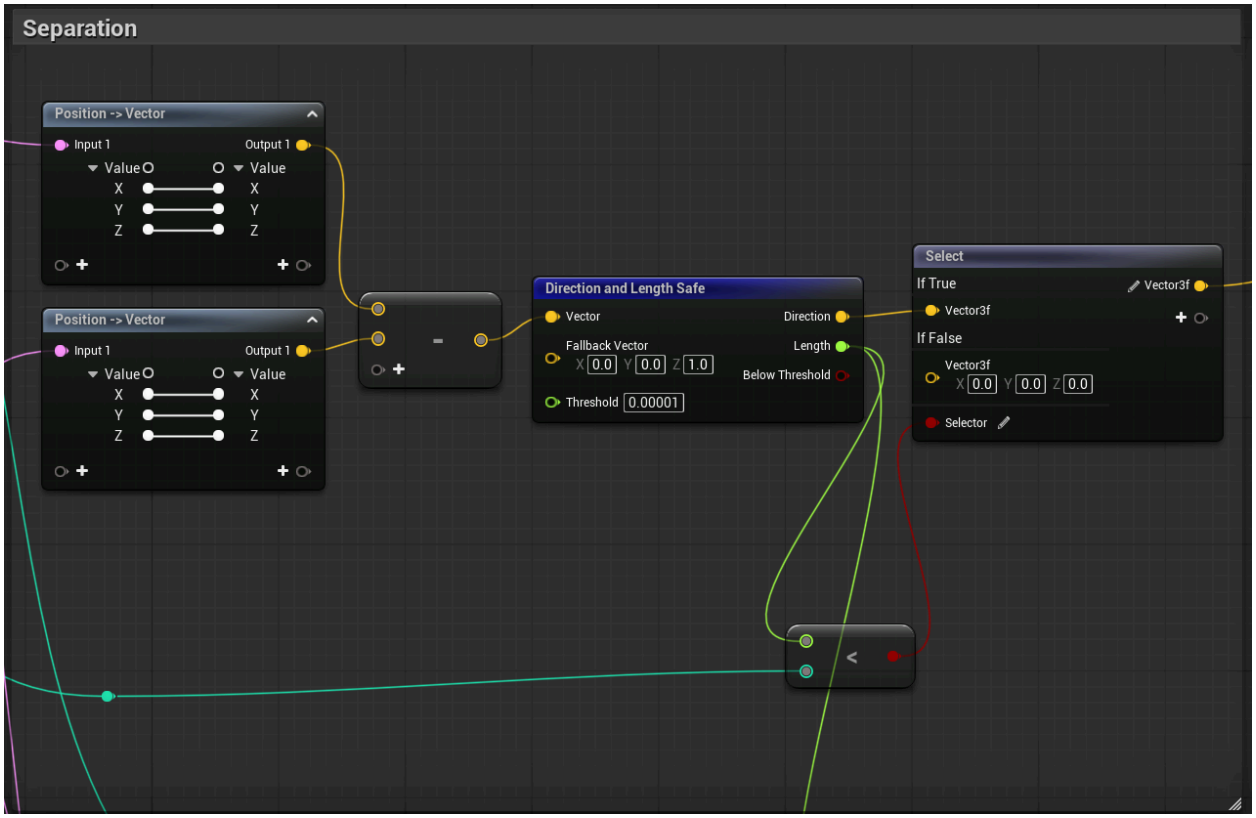
For our first section, we’ll be getting the attributes of each particle in the system so we can accumulate the parameters of the current parameter we are operating on. Remember that this module happened every frame for every particle. Add the following modules and wire them as seen here:



Make sure to search for these Get Position and Get Vector nodes while dragging off of the Particle Attribute Reader from the Map Get Node.

This Map For Index node just returns the index we are currently on from the Map For node, which we wired the number of particles into. Note we are also getting our alignment vector, the velocity of particle we're iterating over. This is sent to the adder section for later.

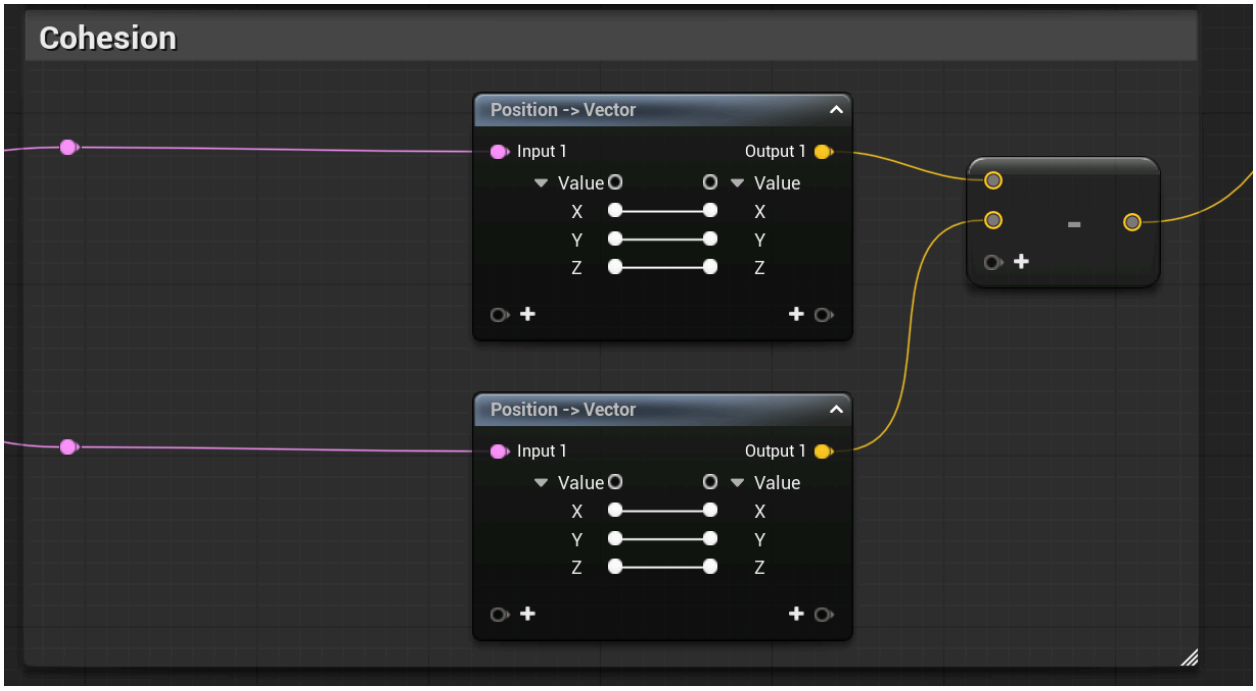
For the next section, add the following nodes and wire them as seen below. In order to get the specific Position -> Vector nodes seen, use the Convert node.



For the top conversion, wire the position from the Map Get node. For the bottom conversion, wire the position acquired through the Get Position by Index node from the last section.

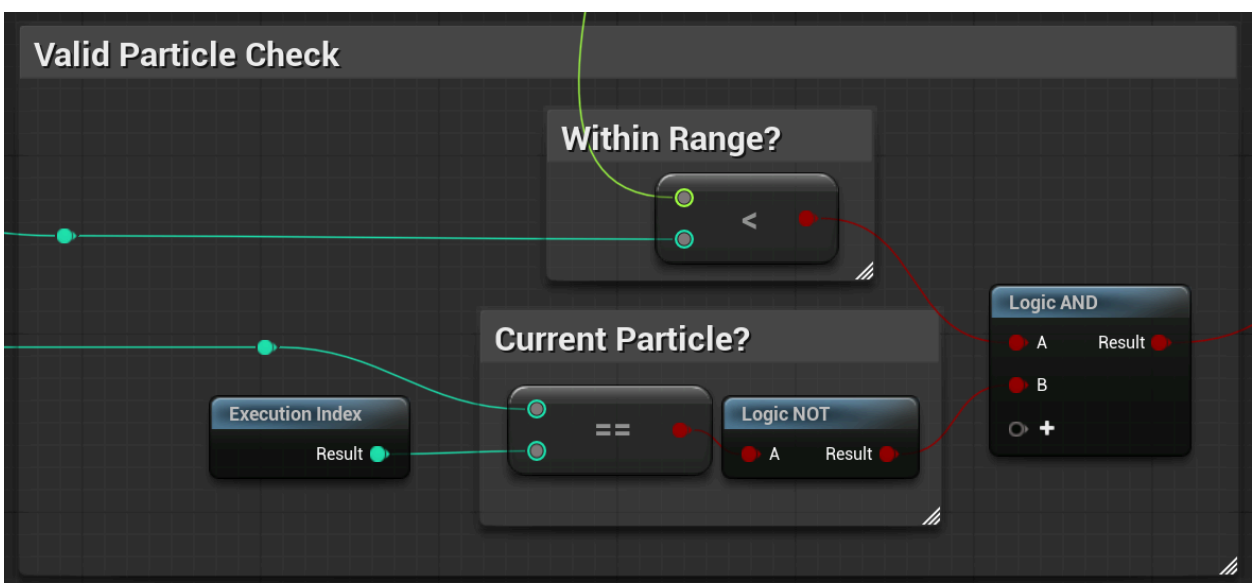
This section calculates the separation vector by finding the vector from the particle we are iterating over to the current particle. It also does a check to see if it's close enough to care by wiring the PersonalSpace input into the less than node, assessing the length of the calculated vector. If the particle is close enough we send that vector over to our adder, otherwise a zero vector is sent.

In the next section, we do something pretty similar.



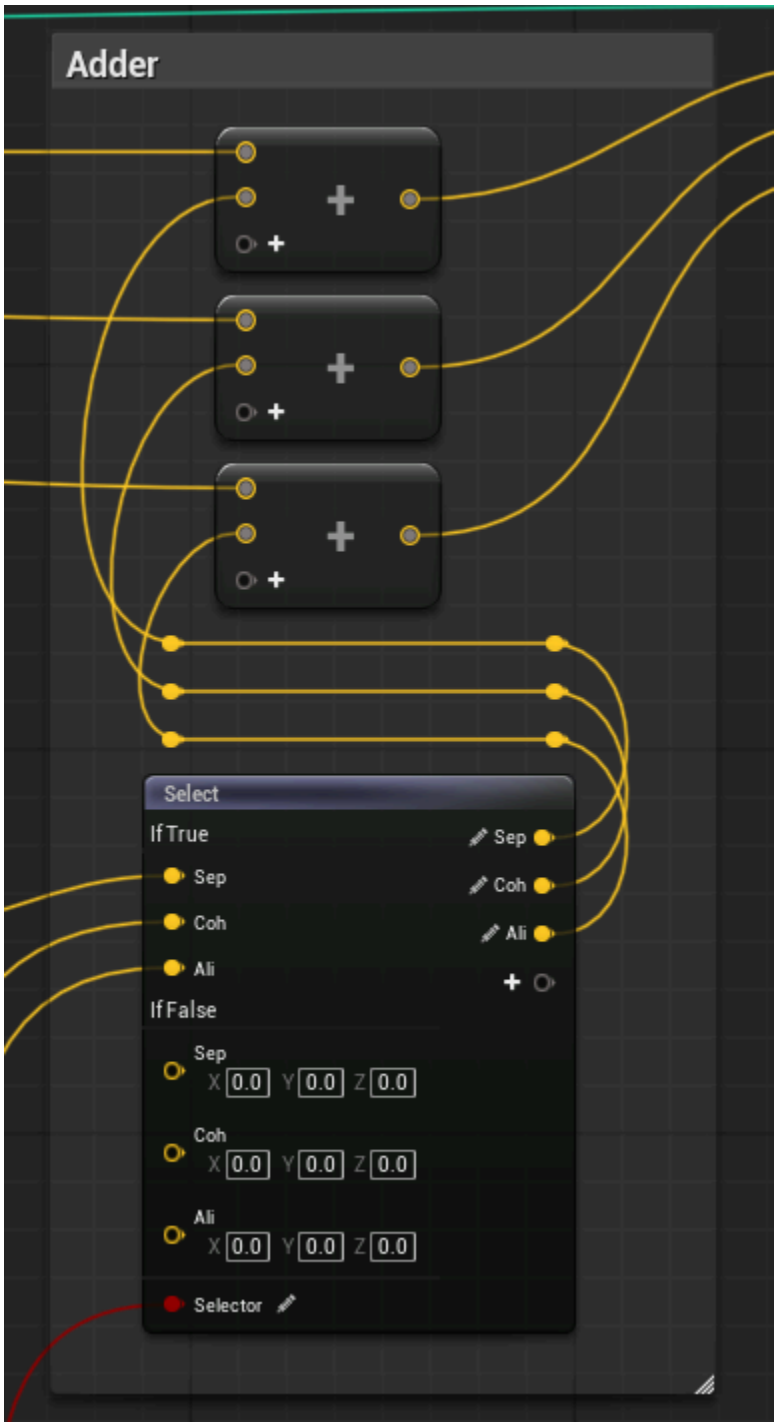
For our cohesion, we also find the vector between the two particles, but it needs to be in the opposite direction. As such, the top covert node takes in the position from the Get Position from Index node from the first section while the bottom node takes in the position from the Map Get node. The output of this operation is sent over to the adder as well.

Before we get to the adder section, we need to do one last thing. We need to see if the particle is within our scanning distance. Unfortunately this cannot be done before, as I've found no way to prevent operation in a scratch module. There's no branches in the traditional sense.



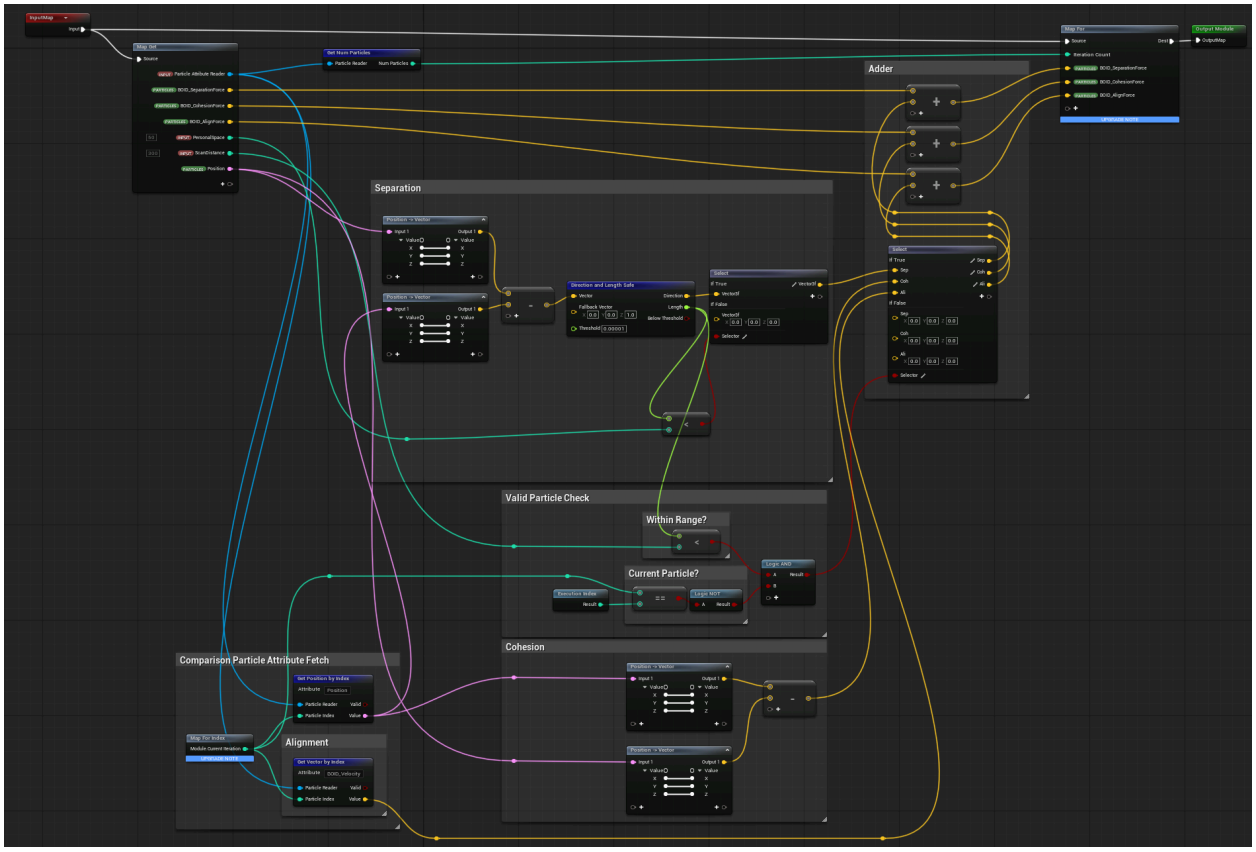
This takes in the distance from the Separation section and checks it against the ScanDistance input from the Map Get node. We also get the iteration index from the Map For node, as seen in the first section, and check it against the Execution Index, a.k.a. The current particle we are accumulating. You can’t separate from yourself. This boolean output is used to drive our adder.

Our adder is pretty simple:

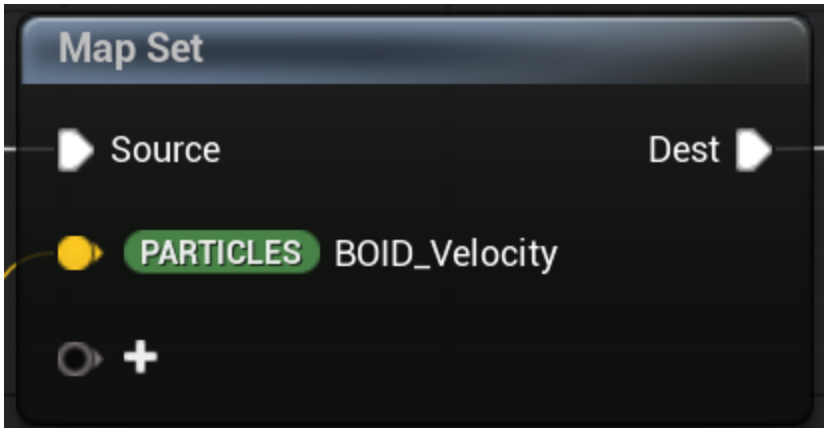


First, it takes in the outputs of our prior sections into the Select node, and based on that boolean either sends the calculated vectors over if within scanning distance or only zero vectors. Each of these output vectors from the Select node are added with the corresponding accumulator from the Map Get node and wired into the output of the Map For node.

This scratch module is complete! This is the most complicated of the three, so pat yourself on the back. Here is the full module wired together:

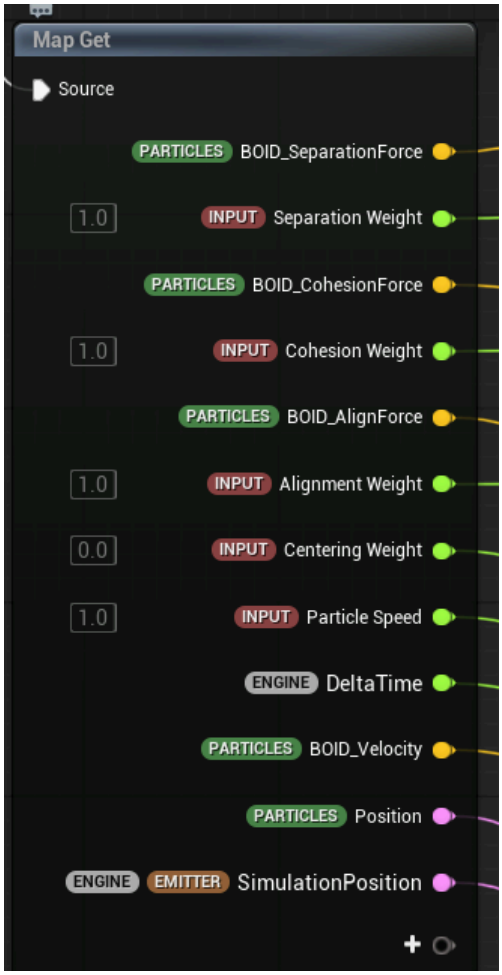


The next scratch module is simpler. Add another to the “Boids” stage and name is “BOID Solve Velocity”. The Map Set node only needs one parameter, BOID_Velocity.

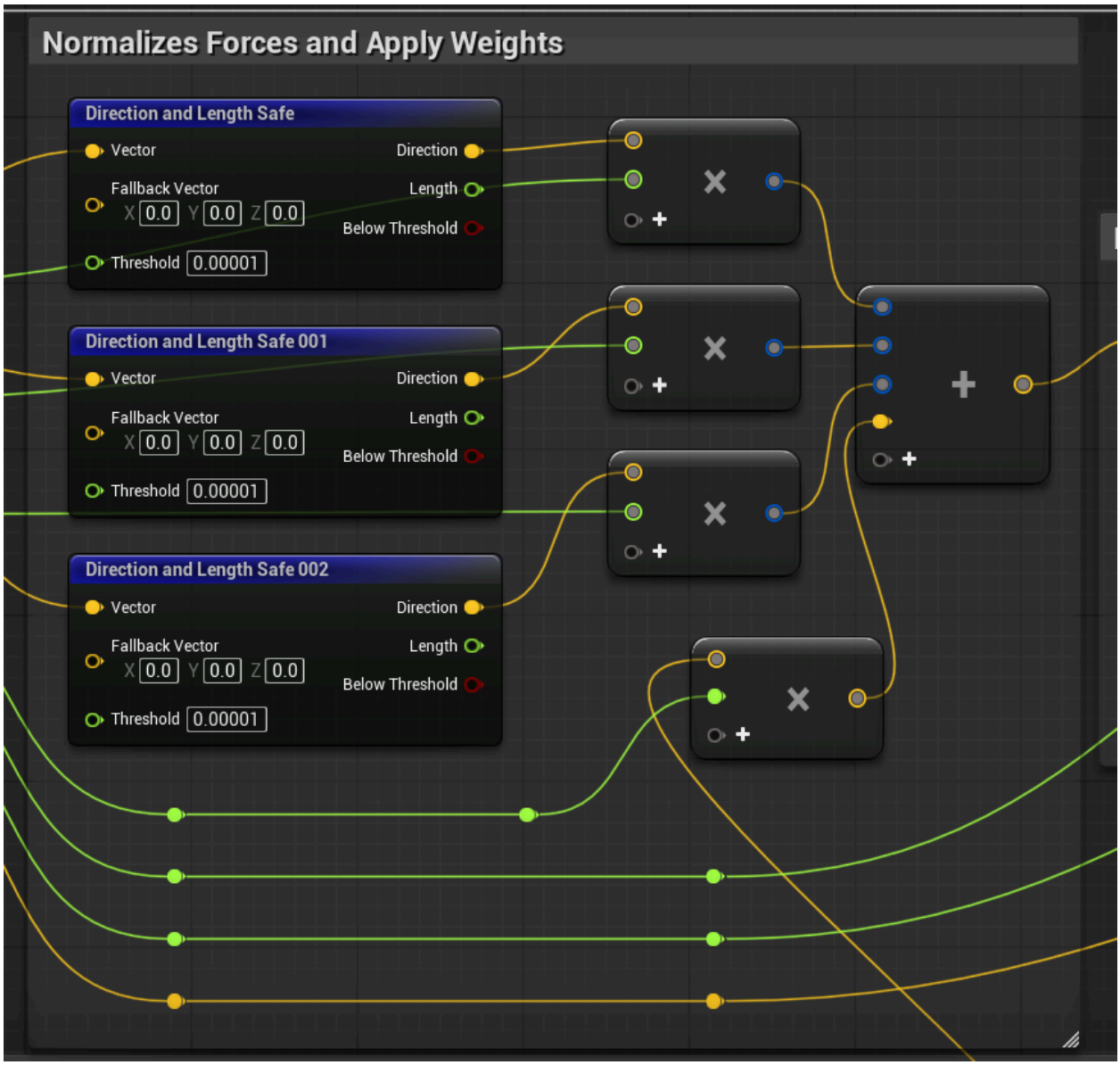


The Map Get nodes is much more full. Before we set it up, add five new float parameter inputs to the “Module Inputs” section of the parameter window. Specifically Alignment Weight, Centering Weight, Cohesion Weight, Particle Speed, and Separation Weight. Give all of these default values of 1.0 apart from Particle Speed which gets a value of 100.0.

Add the following parameters to the Map Get node:



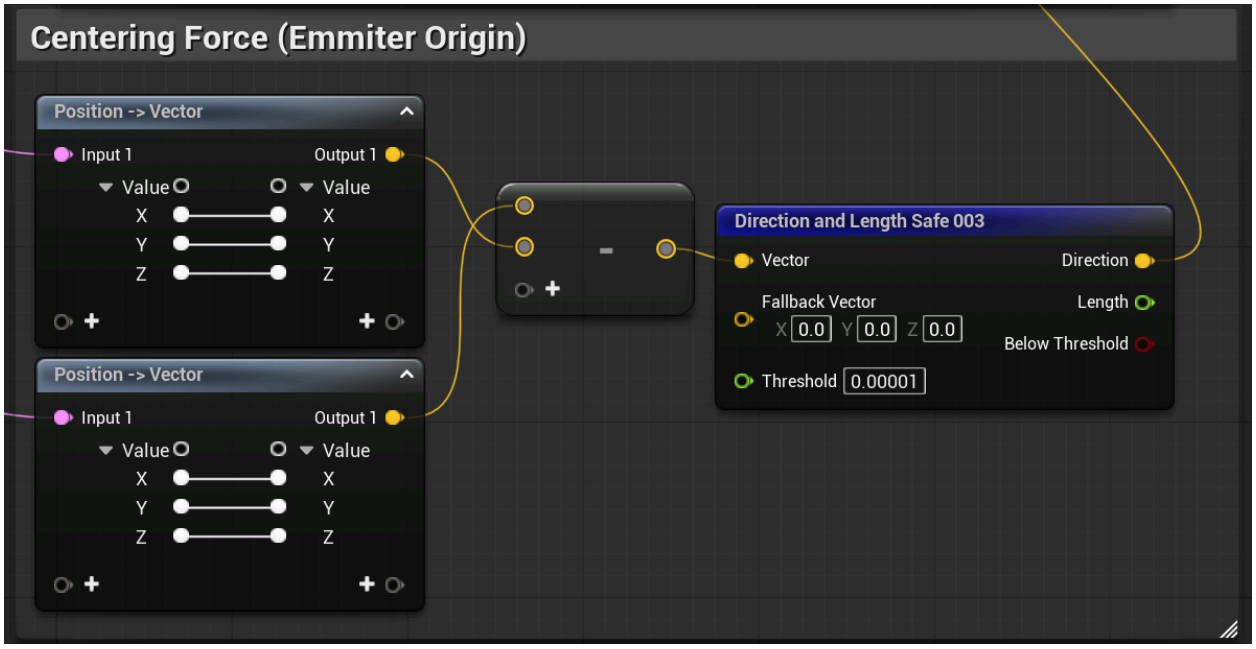
The first section is for normalizing our forces. Set up the node structure below.



This gets a unit vector version of each of our forces, multiplies them by their weight, and adds them together. Note the additionally multiply node wired into the adder, this is for the centering force and weight which is the focus of the next section.

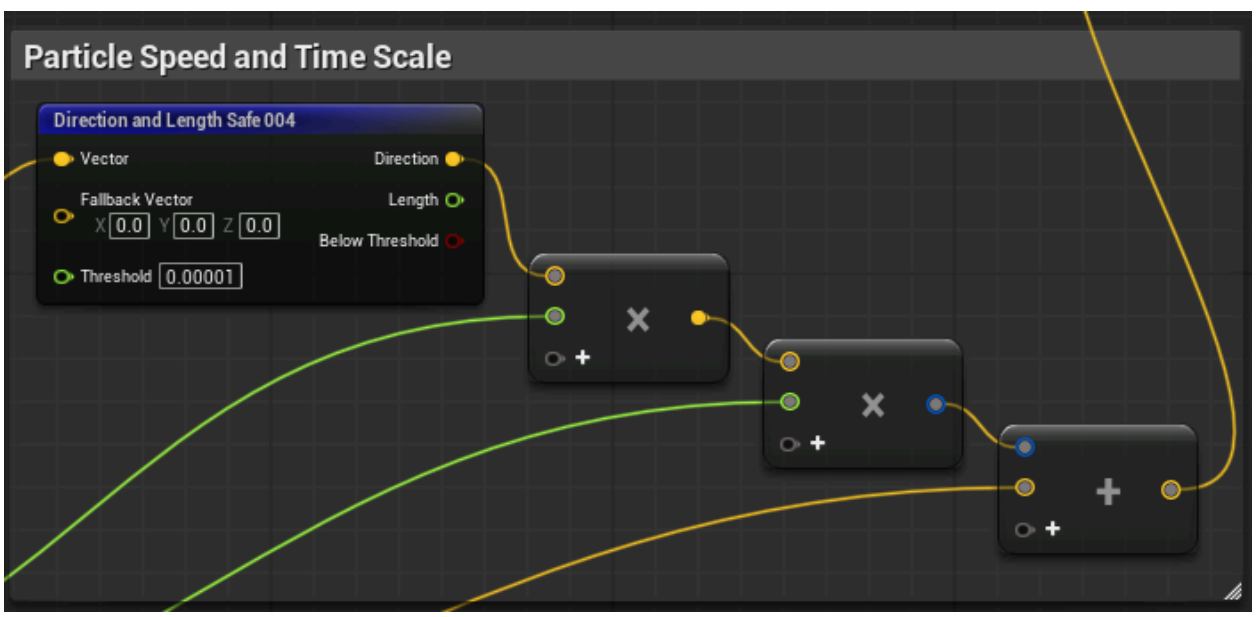
Ignore the reroute nodes below the additional Multiply node, these are for the inputs of the final section.

For the next section, we want to calculate a centering force quickly to ensure our particle simulation doesn't fly off into the sunset.



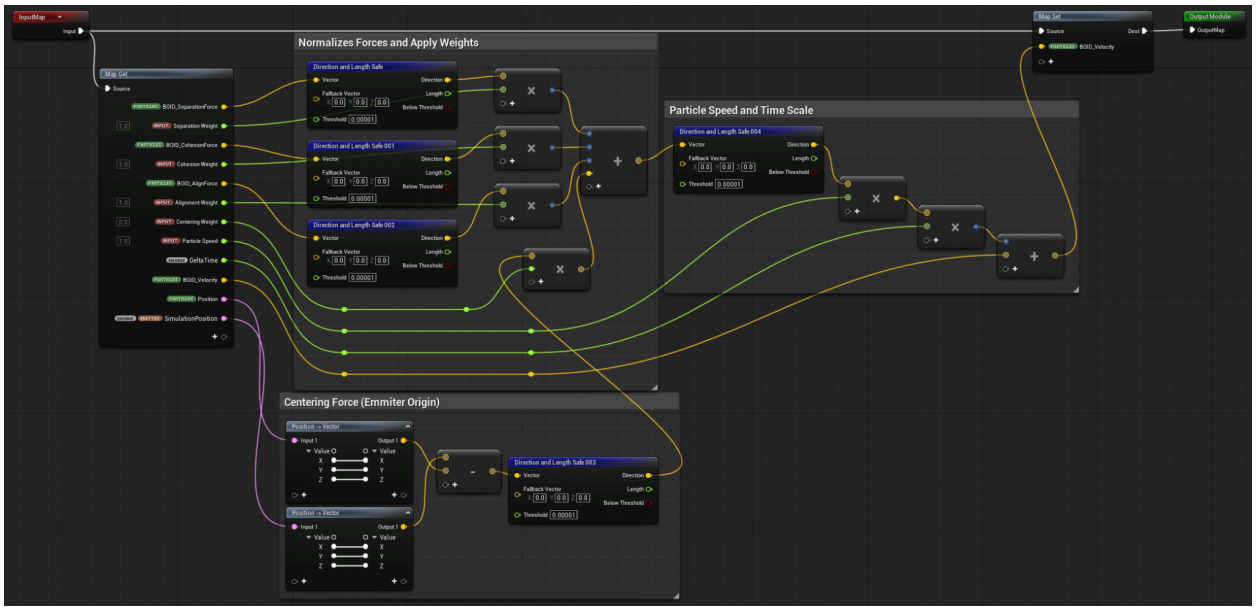
We use the distance between the Particle position (the top Convert node) and the SimulationPosition (the bottom convert node). The output of this section wires into that extra Multiply node from the previous section.

Finally, the last section of this module. We use the last few inputs from the Map Get node, Particle Speed, Delta Time, and BOID_Velocity and set up an operation similar to how acceleration would work.

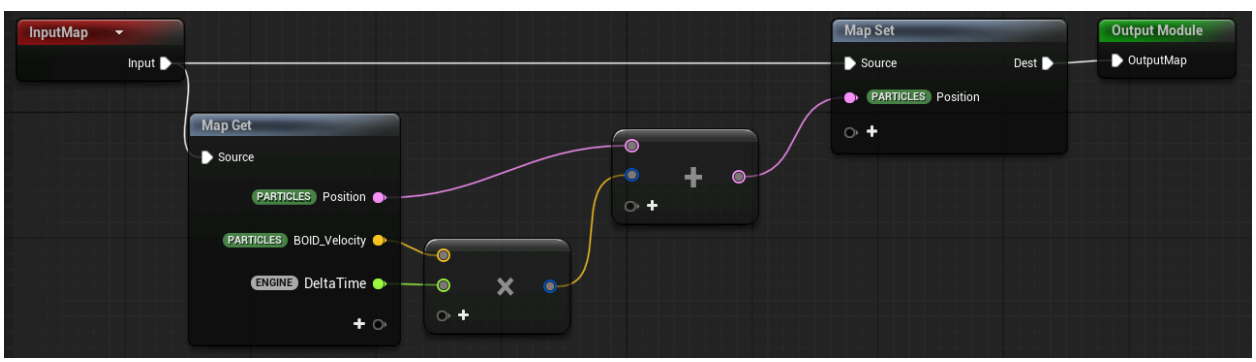


First, the velocity direction is scaled by our intended speed, this gives us a vector that is the

desired velocity. After multiplying it by our time scale we add it to the last frames velocity and set it with the Map Set node. With that, this scratch pad module is complete. Only one more to go. Your finished module should look like this:

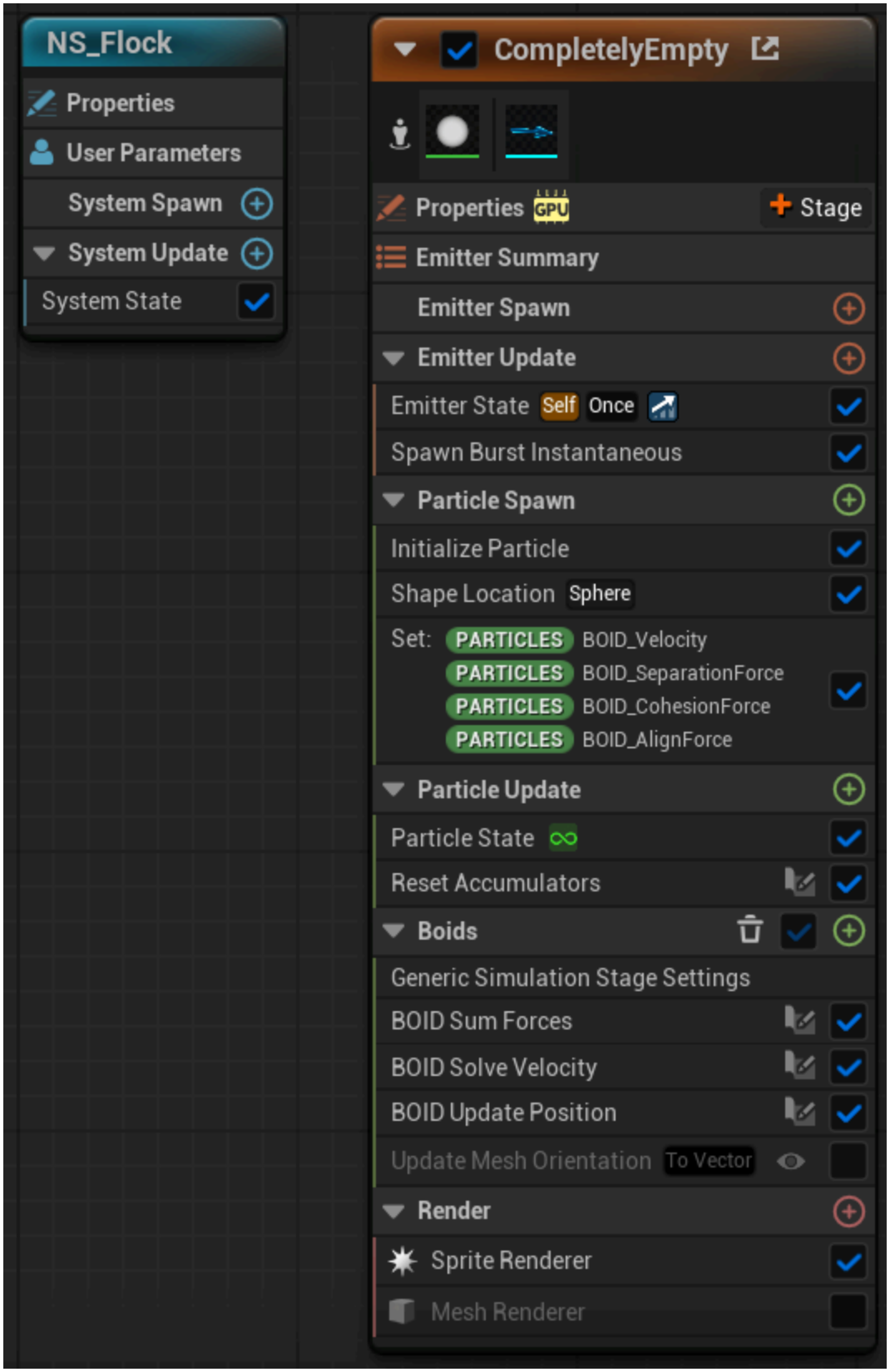


On to the third and final scratch pad module, and the simplest. This one is simple and updates the particle's built-in position parameter using a similar method to the last section we just did.



As you can see above, get the position, calculated velocity, and delta time from the Map Get node, multiply the velocity by the time scale, and add it to our position. Wire the output back into the position parameter in the Map Set node and you're good!

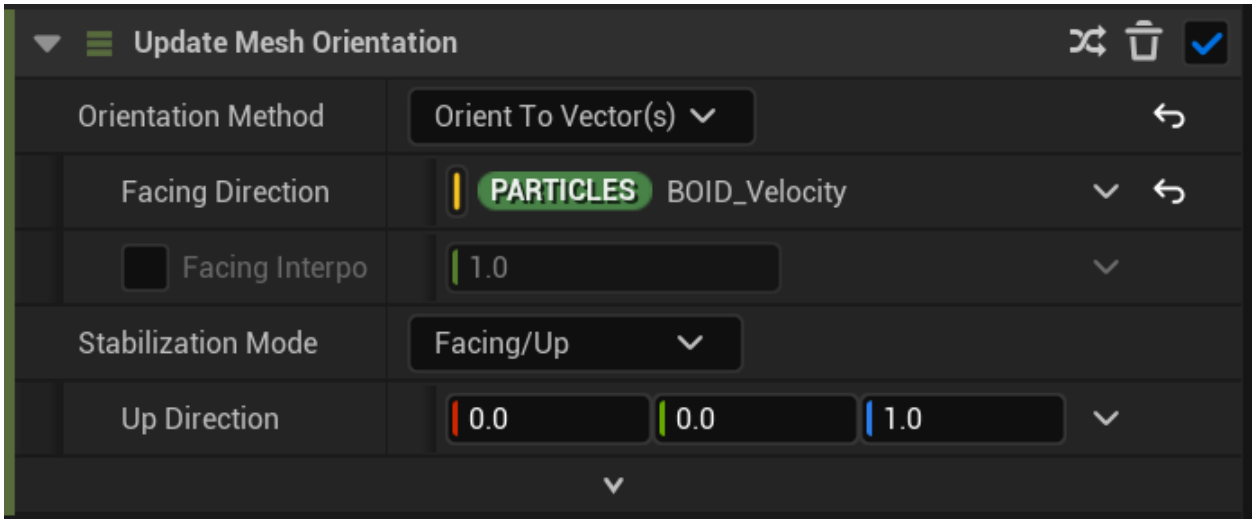
Your simulation should now be complete. If there are any issues, refer to the step through pdf or the blueprint visualizers. Your system overview should look as follows:


NS_Flock
Properties
User Parameters
System Spawn
System Update
System State
CompletelyEmpty
Properties
GPU
Stage
Emitter Summary
Emitter Spawn
Emitter Update
Emitter State
Self
Once
Spawn Burst Instantaneous
Particle Spawn
Initialize Particle
Shape Location
Sphere
Set:
PARTICLES BOID_Velocity
PARTICLES BOID_SeparationForce
PARTICLES BOID_CohesionForce
PARTICLES BOID_AlignForce
Particle Update
Particle State
Reset Accumulators
Boids
Generic Simulation Stage Settings
BOID Sum Forces
BOID Solve Velocity
BOID Update Position
Update Mesh Orientation
To Vector
Render
Sprite Renderer
Mesh Renderer

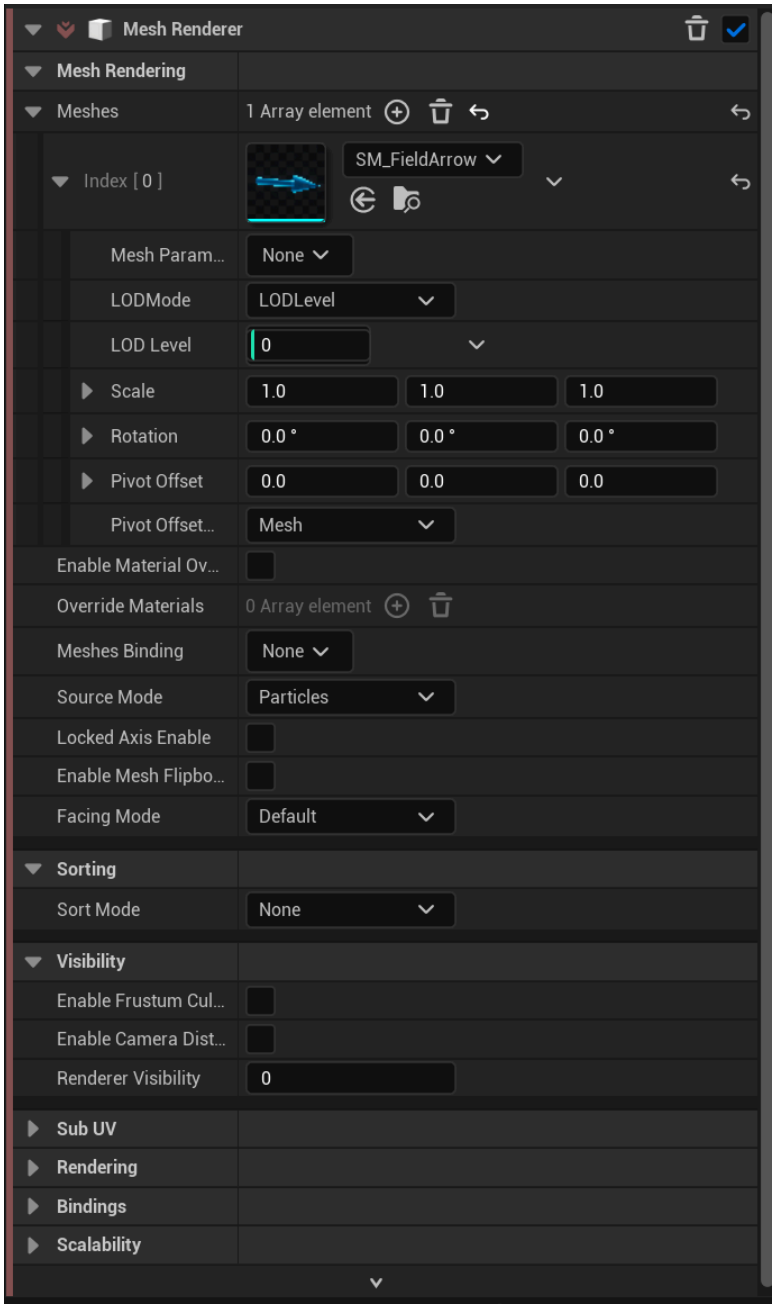
Optional Rendering Solution

As you may have noticed above there are two deactivated modules in the system overview above. These are used to achieve an alternate rendering solution that uses meshes, not sprites. Additionally, with the module “Update Mesh Orientation” we can keep our mesh from twisting and turning erratically, eventually ending up upside down.

Both of these modules are default modules within Niagara. Below are screenshots of their parameters for your reference. This will result in your simulation using blue transparent arrows that point in the direction they’re moving:



Here, only Orientation Method and Facing Direction are modified to Orient To Vector(s) and BOID_Velocity respectively. The Stabilization Mode and Up Direction are fine as default.



Here, all that's changed is the mesh. I like the SM_FieldArrow, which is included with Unreal Engine 5.7.4

Closing

Thanks for following along! Enjoy the configurable boid simulation, there's a lot you can do with it.